Sahte EDM Ölçümünde Fırlatma Yöntemi: Makaledeki Yöntem ile Donmuş Spin Yönteminin Karşılaştırılması

Hazırlayan: Claude (Anthropic yapay zekâ asistanı)

13 Kasım 2026

Özet

Sahte EDM'yi C++ izleyicide ölçmek için parçacığın nasıl fırlatıldığını iki yöntemle karşılaştırıyorum. *Eski yöntem* (makaledeki): parçacık kapalı yörüngenin üzerinden ve fırlatma noktasında momentum büyüklüğü tam sihirli değerde ($|p| = p_{\text{sihirli}}$) fırlatılır. *Yeni yöntem*: parçacığın enerjisi, yatay spin presesyonu sıfır olacak biçimde ayarlanır ($\omega_h = 0$, "donmuş spin"), yani deneyin kendisinin sabit tuttuğu koşulla. İki yöntem aynı kapalı yörünge aramasını ve aynı eğim kestirimini kullanır; tek fark, hangi enerjide ölçüldüğüdür. Bulgular: (1) eski yöntemin sonucu, aynı fizik için fırlatma noktasına göre -456 ile $+2729\times$hedef arasında değişir, yeni yöntemin sonucu %0.2 içinde aynıdır; (2) nedeni, sert kenarlı saptırıcı modelinde $|p|$'nin aynı kapalı yörünge boyunca hücreden hücreye $\sim 10^{-7}$ değişmesidir; (3) eski yöntemin hatası fırlatma anındaki kapalı yörüngenin büyüklüğüyle orantılıdır, bu yüzden makalenin iki demetli sonuçları (yörünge zaten küçük) neredeyse etkilenmez, ama düzeltilmemiş ya da tek demetle düzeltilmiş durumlar 1.1–35 kat yanlış (çoğunlukla abartılmış) ölçülmüştür; (4) σ^2 ölçekleme testi bu hatayı yakalayamaz, çünkü hata da σ^2 ile ölçeklenir. Simülasyonlar yalnız yöntemleri karşılaştırmak için seçildi; tam kapsamlı değildir.

İçindekiler

1 Ne ölçüyoruz ve fırlatma neden önemli? 3

2 Simülasyonun geometrisi 3

3 Eski yöntem (makaledeki) 4

3.1 Adımlar . . . 4

3.2 Kod . . . 4

4 Yeni yöntem (donmuş spin) 5

4.1 Fikir . . . 5

4.2 Adımlar . . . 5

4.3 Kod . . . 5

4.4 Yan yana . . . 6

5 Karşılaştırma testleri 6

5.1 Test 1: Fırlatma noktasından bağımsızlık . . . 7

5.2 Test 2: Neden? Aynı yörüngede $|p|$ hücreden hücreye değişiyor . . . 7

5.3 Test 3: Sahte EDM enerjiye ne kadar duyarlı? . . . 7

5.4 Test 4: Yeni yöntem başlangıç kuralından bağımsız mı? 8
5.5 Test 5: σ^2 ölçekleme testi bu farkı neden göremez? 9
5.6 Test 6: Makalenin Şekil 4'ü, aynı desenlerle 9
5.7 Test 7: Önceki raporlardaki ana sayılar 11

6 Hangisi doğru, ve ne kadar emin olabiliriz? 11

7 Makale için sonuç 12

A Yeniden üretim 12

1 Ne ölçüyoruz ve fırlatma neden önemli?

Ölçülen büyüklük. Sahte EDM, spinin düşey bileşeninin büyüme hızıdır: $f = \mathrm{d}S_y/\mathrm{d}t$. Hedef $f \approx 1$ nrad/s'dir. C++ izleyici bir protonu ve spinini halkada birkaç yüz tur izler; her turda belirli bir noktada S_y kaydedilir ve bu dizinin eğimi ölçülür.

Kapalı yörünge üzerinden fırlatma (iki yöntemde ortak). Kuadrupol kaçıklıkları kapalı yörüngeyi ideal eksenden kaydırır. Parçacık ideal eksenden fırlatılırsa kapalı yörünge etrafında betatron salınımı yapar; bu salınımın S_y'ye katkısı aradığımız seküler eğimden yaklaşık bin kat büyüktür. Bu yüzden iki yöntemde de önce 4 boyutlu kapalı yörünge (x, x', y, y') aranır ve parçacık onun üzerinden fırlatılır [BELGELENMIŞ].

Eğim kestirimi (iki yöntemde ortak). $S_y(t)$'ye "doğru + salınımlar" modeli fit edilir: $S_y(t) = a + bt + \sum_k [c_k \cos(2\pi f_k t) + d_k \sin(2\pi f_k t)]$; yalnız b raporlanır. Düz doğru fiti kullanılmaz, çünkü kalan küçük salınımlar eğimi bozar.

Fırlatmada açık kalan tek seçim: enerji. Kapalı yörünge konumu ve açısı belirler; ama parçacığın *enerjisini* (momentum büyüklüğünü) belirlemez. Her enerji için ayrı bir kapalı yörünge vardır. İki yöntem tam olarak burada ayrılır. Bu önemsiz bir ayrıntı değildir: sahte EDM enerjiye çok duyarlıdır (Bölüm 5.3).

2 Simülasyonun geometrisi

Bir FODO hücresi sekiz elemandan oluşur ve C++ izleyici her turu *sürüklenme (drift)* elemanı 1'den başlatır:

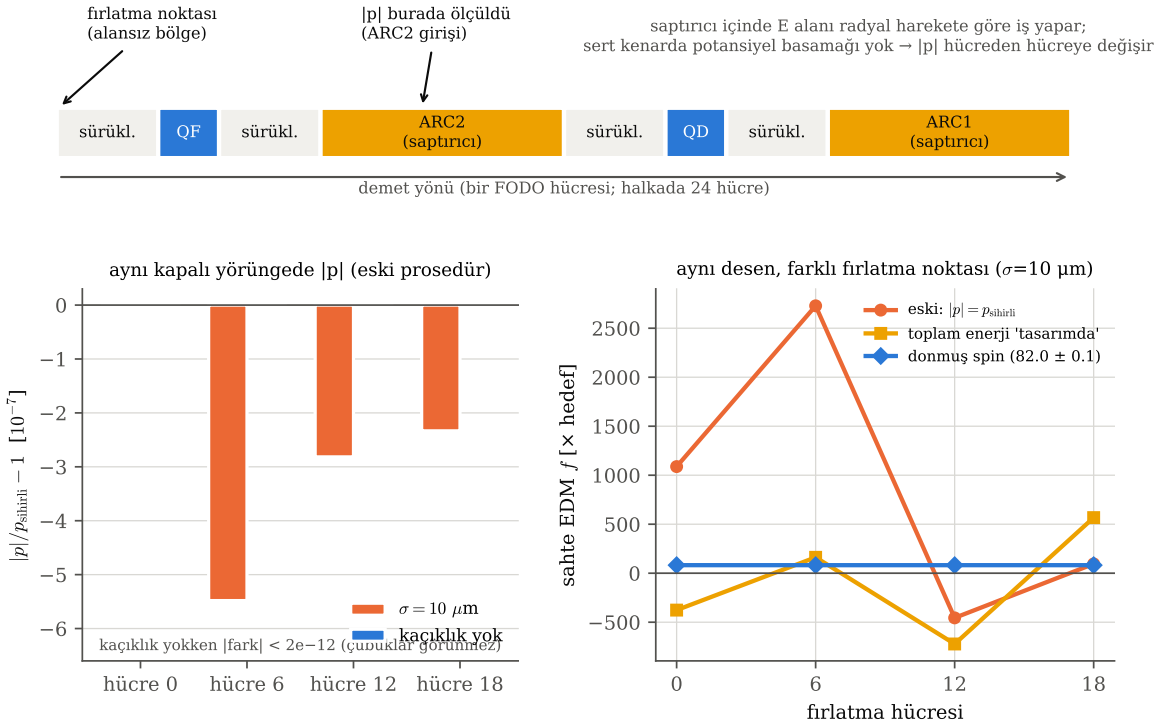
```
// integrator.cpp, run_integration
// Start each revolution from the first drift element (elem =1),
// then wrap around: 1,2,3,4,5,6,7,0.
const int start_elem = 1;
...
if (elem == 0 || elem == 4) { type = 0; target_val = Phi_def; }        // ARC1, ARC2:
↪  saptırıcı
else if (elem == 1 || elem == 3 || elem == 5 || elem == 7) { type = 1; }  // sürüklenme
else if (elem == 2) { type = (current_fodo == 0) ? 4 : 2; }               // QF
else if (elem == 6) { type = 3; }                                          // QD
```

Yani fırlatma noktası ARC1 saptırıcısının hemen çıkışı, QF'ten öncedir: **alansız bir bölge**.

Saptırıcıların alanı sert kenarlıdır:

```
if (element_type == 0) {
    // DEFLECTOR: cylindrical capacitor, hard-edge (no fringe fields).
    ...
    if (n == 1.0) { E_r = E0 * R0 / R; }
```

Alan saptırıcının içinde $E_r = E_0 R_0/R$, dışında sıfırdır; sınırda saçak alanı ve potansiyel basamağı yoktur. Bunun sonucu Bölüm 5.2'de önemli olacak.



Şekil 1: Üst: bir FODO hücresinin eleman sırası ve fırlatma noktası. Sol alt: aynı kapalı yörüngenin farklı hücrelerindeki $|p|$ (Bölüm 5.2). Sağ alt: aynı desen, üç fırlatma kuralı, dört farklı fırlatma noktası (Bölüm 5.1).

3 Eski yöntem (makaledeki)

3.1 Adımlar

1. Parametreler `params.json`'dan okunur; sihirli momentum p_{sihirli} hesaplanır.
2. 4 boyutlu kapalı yörünge aranır: fırlatma noktasında $|p| = p_{\mathrm{sihirli}}$ olan bir parçacığın betatron salınım varyansı Newton yöntemiyle en küçük yapılır.
3. Parçacık bulunan (x, x', y, y') noktasından, **momentum büyüklüğü** p_{sihirli} ve spin boylamsal olarak fırlatılır.
4. 3×10^{-4} s (yaklaşık 67 tur) izlenir; S_y'nin eğimi model fitiyle ölçülür.

Sabit tutulan: fırlatma noktasındaki momentum büyüklüğü.

3.2 Kod

Makalenin Şekil 4'ünü üreten `kmod_drivers/twobeam_oc.py` her ölçüm için şunu çağırır:

```
# kmod_drivers/twobeam_oc.py
def _C(dx, dy, dG =None, tilt =None):
    fcw, _ = fast_measure(dx, dy, tilt=tilt, direction=+1, signed=True, n_turns=28,
    ↪  n_iter=4, dG=dG)
    fccw, _ = fast_measure(dx, dy, tilt=tilt, direction=-1, signed=True, n_turns=28,
    ↪  n_iter=4, dG=dG)
    return fcw / TARGET, fccw / TARGET
```

`fast_measure`'ın çekirdeği (`kmod_drivers/fast_est.py`):

```
fields, y0, beta0, R0, p_mag, direction = setup_fields(cfg)        # p_mag = p_sihirli
T_rev = _T_rev(fields, beta0, R0)
v_co, resid = find_co_4d(fields, p_mag, direction, dx, dy, tilt, T_rev,
                         n_turns =n_turns, n_iter =n_iter, dG =dG)   # adım 2
launch = _make_state(v_co, p_mag, direction, [0.0, 0.0, direction])  # adım 3
fields.poincare_quad_index = 0.0
_, poin, pt = integrate_particle(launch, 0.0, t2, DT, fields =fields, ...)   # adım 4
slope = float(measure_dSy_dt_model(poin[:, 7], pt))
```

Fırlatma durumunu kuran fonksiyon (`berry_data/false_edm_mode_scan.py`); momentum büyüklüğü doğrudan `p_mag`'dir:

```
def _make_state(v, p_mag, direction, spin):
    """(x0, y0, x', y') faz-uzayı noktasından 9-bileşenli yerel başlangıç durumu."""
    return [v[0], v[1], 0.0,
            p_mag * direction * v[2], p_mag * direction * v[3], p_mag * direction,
            spin[0], spin[1], spin[2]]
```

Kapalı yörünge araması (`berry_data/false_edm_4d.py`); aranan büyüklük her turda aynı noktadaki konumun varyansıdır ve aramada da aynı `_make_state` kullanılır:

```
def find_co_4d(fields, p_mag, direction, dx, dy, tilt, T_rev, n_turns=28, n_iter=2,
↪  dG=None):
    def var4(v):
        st = _make_state(v, p_mag, direction, spin)
        _, poin, _ = integrate_particle(st, 0.0, t_probe, DT, fields =fields, ...)
        return float(np.var(poin[:, 0]) + np.var(poin[:, 1]))
    # 4x4 Hessian ile Newton adımları (n_iter kez)
```

4 Yeni yöntem (donmuş spin)

4.1 Fikir

Gerçek deneyde demetin enerjisi ne momentumla ne toplam enerjiyle sabitlenir; polarimetre spinin yatay yönünü (fazını) ölçer ve bir geri besleme, yatay presesyonu sıfır tutacak biçimde enerjiyi ayarlar ("donmuş spin"). JEDI işbirliği COSY'de presesyon fazını geri beslemeyle kilitlemeyi göstermiştir. Yeni yöntem simülasyonda aynı koşulu kurar: sahte EDM, $\omega_h = 0$ olan enerjide ölçülür.

4.2 Adımlar

1. Bir başlangıç kuralıyla ($\delta = 0$) kapalı yörünge aranır ve izlenir: f_0 ve yatay presesyon hızı $\omega_0 = \mathrm{d}/\mathrm{d}t \operatorname{atan2}(S_x, S_z)$ ölçülür.
2. Enerji düzeltilir: $\delta_1 = -\omega_0/(\mathrm{d}\omega_h/\mathrm{d}\delta)$. Eğim $\mathrm{d}\omega_h/\mathrm{d}\delta = -5.238 \times 10^6$ rad/s kaçıklıktan bağımsızdır (Bölüm 5.3). Bu δ_1 ile kapalı yörünge *yeniden* aranır ve izlenir: f_1, ω_1.
3. f ve ω_h, δ'ya doğrusal olduğu için $\omega_h = 0$'daki değer ara değerlemeyle alınır: $f^* = f_0 + (f_1 - f_0)\,(0 - \omega_0)/(\omega_1 - \omega_0)$.

Sabit tutulan: $\omega_h = 0$. Başlangıç kuralı yalnız bir başlangıç tahminidir; sonucu etkilemez (Bölüm 5.4).

4.3 Kod

Tamamı `revizyon/sahte_edm_donmus.py`'dedir. Başlangıç kuralı (enerji korunumlu tahmin + δ):

```
def launcher(delta):
    def make_state(v, p_mag, direction, spin):
        x = v[0]
        W_ref = np.sqrt((p_mag * C_LIGHT) ** 2 + (M_P * C_LIGHT ** 2) ** 2)
        Wk = W_ref - Q_E * (-E0_MAGIC * R0 * np.log((R0 + x) / R0))
        p = np.sqrt(Wk ** 2 - (M_P * C_LIGHT ** 2) ** 2) / C_LIGHT * (1.0 + delta)
        return [v[0], v[1], 0.0, p * direction * v[2], p * direction * v[3], p * direction,
                spin[0], spin[1], spin[2]]
    return make_state
```

Tek bir δ için ölçüm: eski yöntemle aynı `find_co_4d`, `integrate_particle` ve `measure_dSy_dt_model`; tek ek, yatay presesyon hızının da ölçülmesi:

```
def _one(dx, dy, direction, delta, tilt, dG, n_turns, n_iter, t2):
    ms = launcher(delta)
    false_edm_4d._make_state = ms; false_edm_mode_scan._make_state = ms   # aramada da aynı
    ↪ kural
    fields, y0, beta0, R0_, p_mag, d = setup_fields(cfg)
    v_co, resid = find_co_4d(fields, p_mag, d, dx, dy, tilt, _T_rev(...), n_turns=n_turns,
    ↪ n_iter=n_iter, dG=dG)
    launch = ms(v_co, p_mag, d, [0.0, 0.0, d])
    _, poin, pt = integrate_particle(launch, 0.0, t2, dt, fields =fields, ...)
    f = measure_dSy_dt_model(poin[:, 7], t)                               # eskisiyle aynı
    ang = np.unwrap(np.arctan2(poin[:, 6], poin[:, 8] * d))              # yatay spin açısı
    return f, np.polyfit(t, ang, 1)[0], resid                            # f, omega_h
```

Donmuş spin noktası:

```
def measure_frozen(dx, dy, direction=-1, tilt=None, dG=None, n_turns=28, n_iter=4,
↪ t2=3e-4):
    f0, w0, r0 = _one(dx, dy, direction, 0.0, tilt, dG, n_turns, n_iter, t2)
    d1 = -w0 / S_W_DELTA                                    # S_W_DELTA = -5.238e6 rad/s
    f1, w1, r1 = _one(dx, dy, direction, d1, tilt, dG, n_turns, n_iter, t2)
    fs = f0 + (f1 - f0) * (0.0 - w0) / (w1 - w0)           # omega_h = 0'da f
    return dict(f=fs, f0=f0, f1=f1, w0=w0, w1=w1, delta_star=d1 * (0.0 - w0) / (w1 - w0),
    ↪ ...)
```

4.4 Yan yana

	eski (makale)	yeni (donmuş spin)
kapalı yörünge araması	`find_co_4d`	aynı
eğim kestirimi	`measure_dSy_dt_model`	aynı
sabit tutulan	fırlatma noktasında $\lvert p\rvert = p_{\text{sihirli}}$	$\omega_h = 0$ (deneydeki koşul)
izleme sayısı	1 (+ kapalı yörünge araması)	2 (+ iki kapalı yörünge araması)
maliyet	1 birim	≈ 2 birim
ek çıktı	–	ω_0, ω_1, δ^* (donmuş noktanın enerjisi)

5 Karşılaştırma testleri

Bütün testler $\sigma = 10$ μm (rng 3000, seed 0) deseninde ya da makalenin Şekil 4 kurulumunda (5 seed) yapıldı. Ayar: kapalı yörünge araması 28 tur, 4 iterasyon; izleme 3×10^{-4} s.

5.1 Test 1: Fırlatma noktasından bağımsızlık

Halka simetrik olduğu için, aynı kaçıklık desenini halkada k hücre kaydırmak yalnızca fırlatma noktasını değiştirir; fizik aynıdır. Doğru bir yöntem dört fırlatma noktasında aynı sonucu vermelidir (`tests/teshis_enerji_ABC.py`) [BELGELENMIŞ]:

fırlatma hücresi	0	6	12	18
eski: $\lvert p\rvert = p_{\text{sihirli}}$	1088	2729	−456	95
ara deneme: "toplam enerji tasarımda"	−376	162	−722	567
yeni: donmuş spin	**82.04**	**81.90**	**81.89**	**81.94**

(Hedefin katı.) Eski yöntem aynı fizik için −456 ile +2729 arasında değişiyor; yeni yöntem %0.2 içinde aynı. Şekil 1'in sağ alt paneli bunu gösteriyor.

5.2 Test 2: Neden? Aynı yörüngede $|p|$ hücreden hücreye değişiyor

Eski yöntemle hücre 0'dan fırlatılan parçacığın kapalı yörüngesi boyunca, hücre 0, 6, 12, 18'deki $|p|$ doğrudan ölçüldü (`tests/teshis_p_azimut.py`; ölçüm ARC2 girişinde, yani fırlatma noktasıyla arasında yalnız sürüklenme ve kuadrupol olan yerde) [BELGELENMIŞ]:

hücre	0	6	12	18
$\sigma = 10\ \mu$m: $\lvert p\rvert/p_{\text{sihirli}} - 1$	$+2 \times 10^{-12}$	-5.48×10^{-7}	-2.82×10^{-7}	-2.33×10^{-7}
kaçıklık yok	$\sim 10^{-12}$	$\sim 10^{-12}$	$\sim 10^{-12}$	$\sim 10^{-12}$

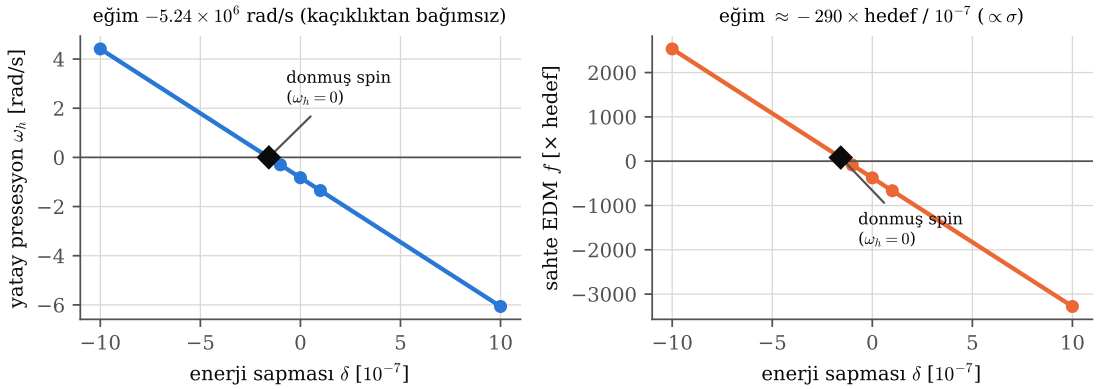
Mekanizma [YORUM]: sert kenarlı saptırıcının içinde elektrik alanı parçacığın radyal hareketine göre iş yapar. Gerçek bir saptırıcıda bu iş, saçak alanındaki potansiyel basamağında geri alınır ve toplam enerji korunur. Sert kenarlı modelde basamak yoktur; bu yüzden kapalı yörüngenin saptırıcı içindeki radyal değişimine bağlı olarak $|p|$ bir hücreden diğerine değişir. Kaçıklık yokken kapalı yörünge ideal eksendir ve değişim sıfırdır.

Sonuç: eski yöntem "hücre k'da $|p| = p_{\text{sihirli}}$" derken, her k için $\sim 10^{-7}$ farklı bir yörünge enerjisi seçer.

Çapraz kontrol [BELGELENMIŞ]: Test 1'deki dört donmuş spin çözümünden, donmuş spin yörüngesinin her hücredeki $|p|$'si hesaplandı; hücre 0'a göre farklar -5.482, -2.816, -2.333×10^{-7}. Doğrudan ölçümle dört basamak aynı. Yani dört farklı noktadan bulunan donmuş spin çözümleri *tek ve aynı yörüngedir*.

5.3 Test 3: Sahte EDM enerjiye ne kadar duyarlı?

Aynı desende enerji $\delta = \pm 10^{-7}, \pm 10^{-6}$ kaydırılarak izlendi [BELGELENMIŞ]:



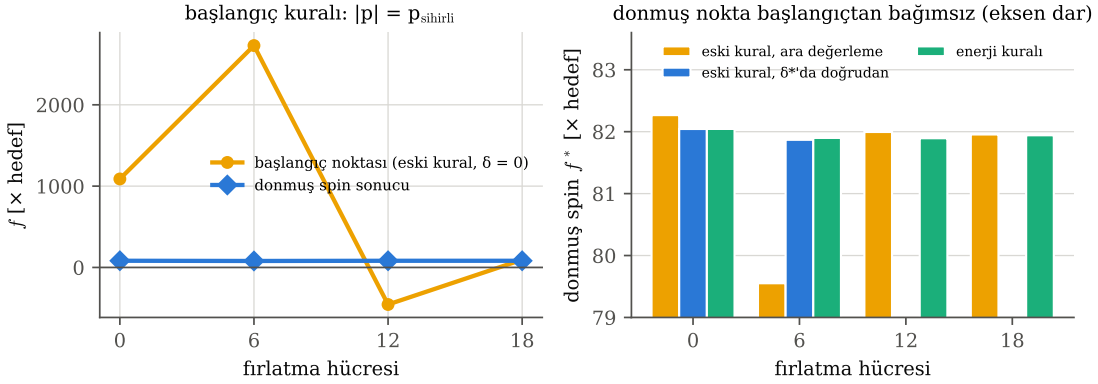
Şekil 2: Enerji taraması. Sol: yatay presesyon; sağ: sahte EDM. İkisi de δ'ya doğrusal; elmas, donmuş spin noktası.

$\delta = 10^{-7}$ başına sahte EDM $\approx 290\times$hedef değişiyor. Test 2'deki $|p|$ farkları (2–5×10^{-7}) bu yüzden yüzlerce hedeflik farklar yaratır. Ayrıca [BELGELENMIŞ] (donmuş spin aracının 9 koşumundan): $\mathrm{d}\omega_h/\mathrm{d}\delta = -5.238\times10^{6}$ rad/s kaçıklıktan bağımsız (halkanın özelliği), $\mathrm{d}f/\mathrm{d}\delta$ ise kaçıklıkla doğrusal ölçekleniyor.

5.4 Test 4: Yeni yöntem başlangıç kuralından bağımsız mı?

Yeni yöntemin bir başlangıç kuralına ihtiyacı var ($\delta = 0$ noktası). Varsayılan kural "enerji korunumlu" tahmindir ve bu, fırlatma noktası alansız bölgede olduğu için fiziksel bir anlam taşımaz. Bu yüzden aynı prosedür, başlangıç kuralı olarak *eski yöntemin kuralıyla* ($|p| = p_{\text{sihirli}}(1+\delta)$) dört fırlatma hücresinde tekrar çalıştırıldı (`tests/teshis_baslangic_kurali.py`) [BELGELENMIŞ]:

fırlatma hücresi	0	6	12	18
başlangıç noktası, eski kural ($\delta=0$): f_0	1088	2729	−456	95
başlangıç noktası: ω_0 [rad/s]	+1.81	−2.48	−0.39	−0.015
ikinci nokta: ω_1 [rad/s]	−0.90	+1.2	+0.19	+0.0075
ara değerleme: f^*	82.27	79.55	82.00	81.96
δ^*'da doğrudan ölçüm: f	**82.04**	**81.87**	–	–
ve orada ω_h [rad/s]	-7×10^{-6}	$+2\times10^{-5}$	–	–
enerji kuralından başlayarak (Test 1): f^*	82.04	81.90	81.89	81.94



Şekil 3: Sol: eski kuraldan başlayınca başlangıç değeri fırlatma noktasına göre çok farklı, ama donmuş spin sonucu aynı. Sağ: iki başlangıç kuralının verdiği donmuş spin sonuçları (dar eksen).

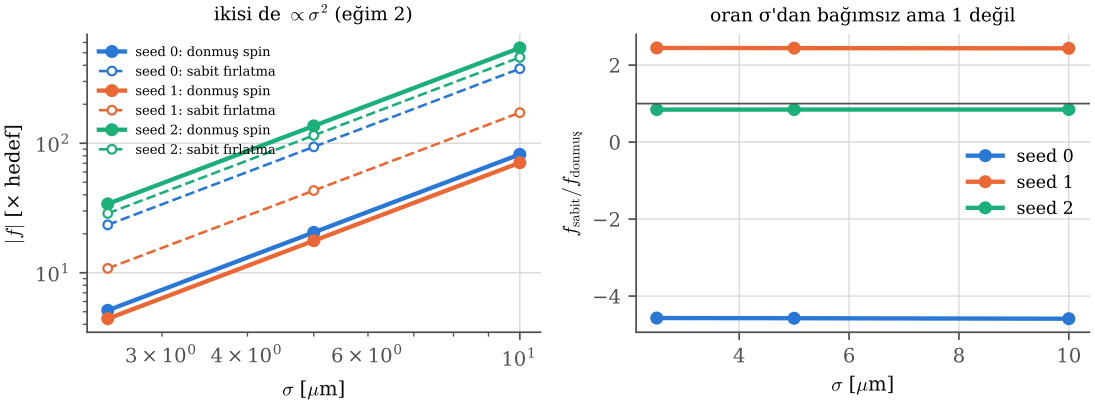
Okunuşu [BELGELENMIŞ]:

1. *Donmuş nokta başlangıç kuralından bağımsız bulunuyor.* Eski kuraldan başlanınca bulunan δ^* değerleri ($+2.313 \times 10^{-7}$, -3.168×10^{-7}, -5.02×10^{-8}, -1.95×10^{-9}), donmuş spin yörüngesinin o hücredeki $|p|$ sapmasıyla (Test 2'nin çapraz kontrolü: $+2.313 \times 10^{-7}$, -3.168×10^{-7}, -5.024×10^{-8}, -1.952×10^{-9}) dört basamak aynı. Yani iki farklı başlangıç kuralı aynı yörüngeyi buluyor.
2. *δ^*'da doğrudan ölçülen f, enerji kuralıyla bulunan değerle %0.04 içinde aynı* (82.04 ve 81.87'ye karşı 82.04 ve 81.90); orada ω_h gerçekten sıfır (10^{-5} rad/s).
3. *İki noktalı ara değerleme ise bu testte %3'e kadar sapıyor* (hücre 6: 79.55). Nedeni: eski kuralda ω_h'nin δ'ya eğimi farklıdır ($\approx -7.8 \times 10^6$ rad/s; aracın varsaydığı -5.24×10^6 enerji kuralına aittir). Bu yüzden ikinci nokta donmuş noktaya düşmez ($|\omega_1| \approx 1$ rad/s) ve ara değerleme geniş bir aralıkta yapılır. Donmuş noktanın dışında f'nin davranışı tam doğrusal olmadığından ara değerleme hata yapar [HIPOTEZ].
4. *Önceki bütün üretim koşumları için sonuç [BELGELENMIŞ]:* enerji kuralıyla başlayan 666 koşumun hepsinde ikinci nokta donmuş noktaya çok yakındı ($|\omega_1|$ en fazla 2.7×10^{-4}, medyanı 10^{-5} rad/s). Bu testteki 0.9–1.2 rad/s ile karşılaştırınca, önceki sonuçlar fiilen donmuş noktada doğrudan ölçülmüş sayılır; ara değerleme hatası ihmal edilebilir.
5. *Araçta yapılan iyileştirme:* $|\omega_1| > 10^{-3}$ rad/s ise araç artık δ^*'da üçüncü bir ölçüm yapıyor ve o değeri raporluyor (`W1_ESIK`). Önceki koşumların hiçbirinde bu eşik aşılmadığı için eski sonuçlar değişmez; araç başlangıç kuralına karşı sağlam hâle gelir.

5.5 Test 5: σ^2 ölçekleme testi bu farkı neden göremez?

Makalede ve önceki çalışmalarda eski yöntem, $\sigma = 2.5$, 5, 10 μm'de $p \approx 2.00$ üssüyle doğrulanmıştı. Yeni yöntem de $p = 2.0002$ veriyor. Ama iki yöntemin değerleri farklı (10 μm'de eski 1089, 58, 1587; yeni 82, -71, 544).

Nedeni [BELGELENMIŞ]: eski yöntemin hatası $= (\mathrm{d}f/\mathrm{d}\delta) \times \delta_{\text{fırlatma}}$. İki çarpan da kaçıklıkla doğrusal: $\mathrm{d}f/\mathrm{d}\delta \propto \sigma$ (Test 3), $\delta_{\text{fırlatma}} \propto x_{\text{CO}} \propto \sigma$ (Test 2). Çarpım $\propto \sigma^2$, yani hata doğru değerle *aynı üsle* ölçeklenir.

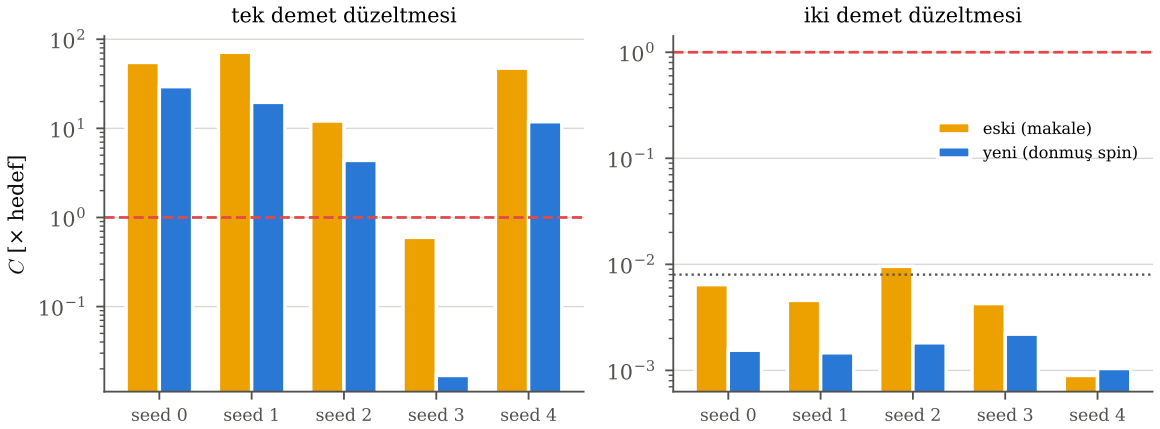


Şekil 4: Sol: sabit başlangıç kuralı ve donmuş spin; ikisi de σ^2. Sağ: oranları her seed için σ'dan bağımsız ama 1'den farklı (-4.6, 2.4, 0.84).

Ders: σ^2 testi mekanizmanın türünü (ikinci mertebe geometrik faz) doğrular; mutlak değeri doğrulamaz. Mutlak değer için Test 1 gibi bir değişmezlik testi gerekir.

5.6 Test 6: Makalenin Şekil 4'ü, aynı desenlerle

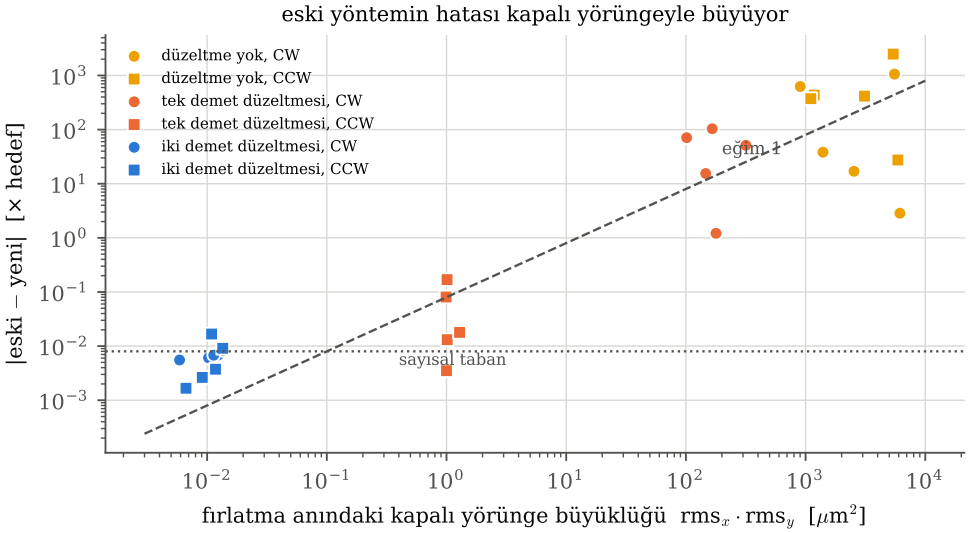
Eski değerler makalenin kendi çıktısıdır (`kmod_drivers/twobeam_oc_14nm.json`); yeni değerler aynı desen, aynı gürültü, aynı düzeltmeyle donmuş spin aracından (`donmus_fig4.jsonl`) [BELGELENMIŞ]:



Şekil 5: Şekil 4 kurulumu (10 μm korelasyonsuz kaçıklık, 14 nm gürültü, `rcond` $= 0.01$). C, seed başına.

	eski			yeni		
durum	f_{CW} medyan	f_{CCW} medyan	C medyan	f_{CW}	f_{CCW}	C
düzeltme yok	-0.08	-313	150	38	78	49
tek demet	-25.6	-1.55	47	-10.1	-1.56	12
iki demet	0.006	0.001	0.0046	0.000	0.002	0.0015

Tablo yanıltıcı olmasın diye her yönü ayrı inceledim. Tek demet düzeltmesinde yalnız CCW yörüngesi düzeltilir; CW yörüngesi büyük kalır. Eski ve yeni yöntem CCW'de neredeyse aynı (fark ≤ 0.17), CW'de çok farklı (fark 1–100) [BELGELENMIŞ]. Bu, hatanın kapalı yörüngenin büyüklüğüne bağlı olduğunu düşündürür; doğrudan sınadım:



Şekil 6: Eski ve yeni yöntem arasındaki fark, fırlatma anındaki kapalı yörüngenin büyüklüğüne (yatay ve düşey rms'in çarpımı, R matrisinden) karşı. Her nokta bir seed ve bir yön.

Okunuşu [BELGELENMIŞ]: fark, kapalı yörüngenin büyüklüğüyle birlikte yaklaşık doğrusal büyüyor (kesikli çizgi eğim 1). Bu, Test 5'teki açıklamayla tutarlı: hata $\propto \sigma \cdot \sigma$, kapalı yörünge büyüklüğü (rms$_x$·rms$_y$) de $\propto \sigma^2$. Üç rejim:

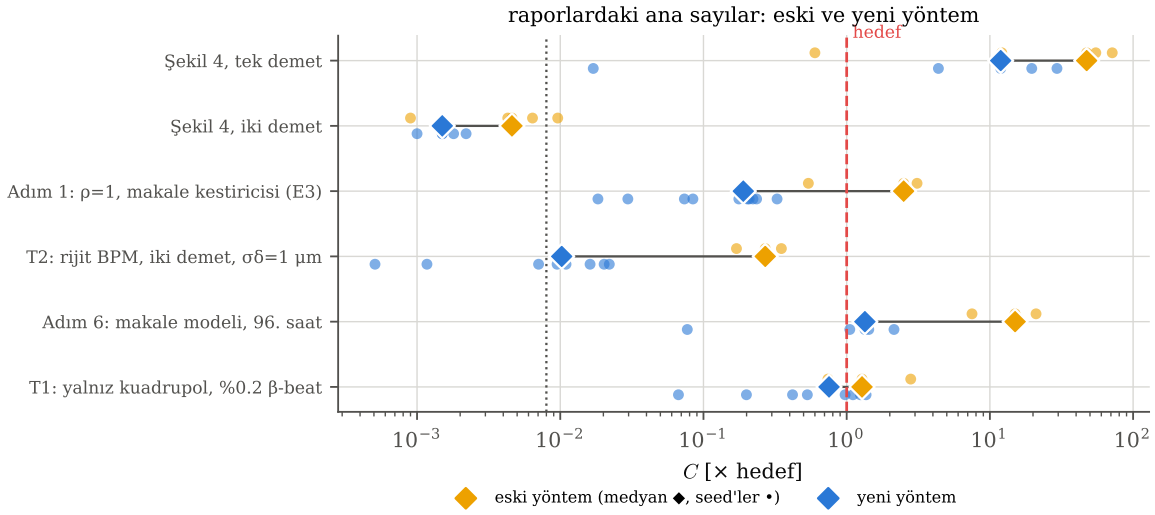
- *İki demet düzeltmesi* (yörünge ~ 0.01 μm^2): fark ≤ 0.02, yani sayısal taban; iki yöntem uyuşuyor.
- *Tek demet, düzeltilen CCW yönü* (~ 1 μm^2): fark ≤ 0.17.

- *Tek demet CW yönü ve düzeltilmemiş desenler* (10^2–10^4 μm^2): fark 1–3000. Düzeltilmemiş desenlerde eski yöntem C'yi 1.1–13 kat yüksek veriyor (medyan 150'ye karşı 49) ve bir seed'de CW işareti farklı (-0.08'e karşı $+38$).

Bir incelik: C iki yönün farkıdır. Eski yöntemin hatası iki yönde farklıdır, çünkü iki yönün kapalı yörüngeleri farklıdır (kuadrupol tekmeleri yönle işaret değiştirir). Bu yüzden hata C'de sadeleşmez; tek demet düzeltmesinde olduğu gibi bir yönün yörüngesi büyük, diğerininki küçükse hata doğrudan C'ye geçer [YORUM].

5.7 Test 7: Önceki raporlardaki ana sayılar

Revizyon kontrolleri sırasında eski yöntemle elde edilip yeni yöntemle tekrarlanan sayılar [BELGELENMIŞ]:



Şekil 7: Her satır bir sonuç; turuncu eski, mavi yeni yöntem. Elmaslar medyan, noktalar seed'ler. Eski değerler 3–5 seed, yeniler 5–10 seed.

sonuç	eski medyan	yeni medyan	yorum
Şekil 4, iki demet	0.0046	0.0015	ikisi de tabanda; değişmedi
Şekil 4, tek demet	47	12	4 kat abartılmıştı
Adım 1: $\rho = 1$, makale kestiricisi	2.5	0.19	hüküm değişti: hedefin altında
T2: rijit BPM, iki demet, $\sigma_\delta = 1$ μm	0.27	0.010	27 kat abartılmıştı
Adım 6: makale modeli, 96. saat	15	1.3	"çalışmaz" $\rightarrow$ "marjsız"
T1: yalnız kuadrupol, %0.2 β-beat	1.3	0.75	aynı yönde, 2 kat

Desen tutarlı [YORUM]: kalan kapalı yörünge küçükse (iki demet, trim) iki yöntem uyuşuyor; büyükse (tek demet, yanlış BPM modeli, düzeltilmemiş) eski yöntem sahte EDM'yi ya abartıyor ya da yanlış işaretle veriyor. Hükmü değişen iki sonuç var: Adım 1'de makale kestiricisi $\rho = 1$'de hedefin altına iniyor; Adım 6'da makale modeli "çalışmaz" değil "marjsız".

6 Hangisi doğru, ve ne kadar emin olabiliriz?

Yeni yöntemden yana kanıtlar [yorum].

1. *Deneyle aynı çalışma noktası:* deney $\omega_h = 0$'ı sabit tutar; yeni yöntem de.
2. *Değişmezlik:* fiziksel olarak aynı durum (farklı fırlatma noktası) aynı sonucu veriyor (Test 1); eski yöntem vermiyor.

3. *Tek yörünge:* dört noktadan bulunan çözümler aynı yörünge (Test 2 çapraz kontrolü).
4. *Başlangıçtan bağımsızlık:* başlangıç kuralı değişse de sonuç aynı (Test 4).

Eski yöntem "yanlış fizik" mi? Hayır [YORUM]. Eski yöntem doğru bir izleme yapıyor, ama kontrol edilmeyen bir enerjide. Ölçtüğü değer, o enerjideki sahte EDM'dir; o enerji ise fırlatma noktasına ve kapalı yörüngeye bağlıdır. Sorun ölçümde değil, çalışma noktasının tanımındadır.

Saçak alanlı (gerçekçi) bir modelde eski yöntem doğru olur muydu? [hipotez] Kısmen. Saçak alanı olan bir saptırıcıda toplam enerji korunur ve alansız bölgelerin hepsinde $|p|$ aynıdır; Test 1'deki fırlatma noktası bağımlılığı büyük ölçüde kaybolurdu. Ama kaçıklık, kapalı yörüngenin uzunluğunu ve saptırıcılardaki ortalama yarıçapını değiştirir; bu da $\omega_h = 0$ olan enerjiyi tasarım enerjisinden kaydırır. Eski yöntem tasarım enerjisinde ölçeceği için yine donmuş noktanın dışında kalırdı. Bu kaymanın büyüklüğünü bu modelde ölçemiyorum (sert kenar onu fırlatma noktası etkisiyle karıştırıyor). Her iki durumda da deneyle eşleşen tanım $\omega_h = 0$'dır.

Yeni yöntemin sınırları.

- Sert kenarlı saptırıcı modeli değişmedi; yeni yöntem yalnız bu modelde çalışma noktasını doğru seçer. Saçak alanının kendisinin sahte EDM'ye etkisi test edilmedi.
- $f(\delta)$ ve $\omega_h(\delta)$'nın doğrusallığı tek desende ($\delta = \pm 10^{-6}$) sınandı. İkinci nokta donmuş noktadan uzak düşerse iki noktalı ara değerleme %3'e kadar sapabilir (Test 4); araç artık bu durumda δ^*'da üçüncü bir ölçüm yapıyor. Önceki 666 koşumda ikinci nokta zaten donmuş noktadaydı [BELGELENMIŞ].
- Geri beslemenin dinamiği ve demetin momentum dağılımı ($\sim 10^{-4}$) simüle edilmedi; tek parçacık ortalama δ'yı temsil eder. f ve ω_h δ'ya doğrusal olduğu için ortalama alma bunu değiştirmez [YORUM].
- Maliyet yaklaşık iki katı.
- Eski yöntemin donmuş nokta dışındaki $\mathrm{d}f/\mathrm{d}\delta$ eğimi fırlatma noktasına göre farklı çıkıyor; bunu açıklamadım. Bu, eski değerleri tek bir eğimle "düzeltmeyi" imkânsız kılıyor: eski sayılar yeniden ölçülmelidir, dönüştürülemez.

7 Makale için sonuç

1. Makalenin ana sayısı, iki demetli düzeltmeden sonraki sahte EDM, iki yöntemde de sayısal tabanda (0.0046 ve 0.0015). Ana iddia etkilenmiyor [BELGELENMIŞ].
2. Karşılaştırma için verilen sayılar (düzeltme yok, tek demet) eski yöntemle 1.1–35 kat yanlış [BELGELENMIŞ]. Makalede "iki demet tek demetten $\sim 10^4$ kat iyi" türünden bir oran varsa, yeni yöntemle yaklaşık 8×10^3 olur (12 / 0.0015); oranın büyüklük mertebesi değişmiyor, ama tek demetin kötülüğünün büyük kısmı `rcond` $= 0.01$ seçiminden geliyor (ek rapor, soru 12).
3. Makalede ölçüm prosedürü tanımlanırken "parçacık kapalı yörüngeden, $\omega_h = 0$ olan enerjide fırlatılır" yazılmalı ve kısa bir değişmezlik testi (Test 1) verilmelidir. Bu, hakemin olası "hangi enerjide?" sorusunu önceden cevaplar.
4. Sert kenar modelinin bir sınırlama olarak yazılması önerilir.

A Yeniden üretim

Dal: `claude/brave-allen-92wn61`.

dosya	işlev
`kmod_drivers/fast_est.py`, `berry_data/false_edm_4d.py`, `berry_data/false_edm_mode_scan.py`	eski yöntem (değiştirilmedi)
`revizyon/sahte_edm_donmus.py`	yeni yöntem
`revizyon/tests/teshis_enerji_ABC.py`	Test 1 ve 3
`revizyon/tests/teshis_p_azimut.py`	Test 2
`revizyon/tests/teshis_baslangic_kurali.py`, `revizyon/tests/teshis_baslangic_dogrulama.py`	Test 4
`revizyon/donmus_fig4.py`, `revizyon/firlatma_eski_ham.py`	Test 6
`revizyon/firlatma_sekiller.py`, `revizyon/ek_sekiller.py`	şekiller